MODULE NAME:	MODULE CODE:
PROGRAMMING 1A	PROG5121/p/w

ASSESSMENT TYPE: POE (PAPER & MARKING RUBRICS)

TOTAL MARK ALLOCATION: 300 MARKS

TOTAL HOURS: A MINIMUM OF 30 HOURS IS SUGGESTED TO COMPLETE THIS ASSESSMENT

By submitting this assignment, you acknowledge that you have read and understood all the rules as per the terms in the registration contract, in particular the assignment and assessment rules in The IIE Assessment Strategy and Policy (IIE009), the intellectual integrity and plagiarism rules in the Intellectual Integrity Policy (IIE023), as well as any rules and regulations published in the student portal.

INSTRUCTIONS:

1. ***No material may be copied from original sources, even if referenced correctly, unless it is a direct quote indicated with quotation marks. No more than 10% of the assignment may consist of direct quotes.***
2. ***Make a copy of your assignment before handing it in.***
3. *Assignments must be typed unless otherwise specified.*
4. *Begin each section on a new page.*
5. *Follow all instructions on the PoE cover sheet.*
6. *This is an individual assignment.*

Referencing Rubric

Providing evidence based on valid and referenced academic sources is a fundamental educational principle and the cornerstone of high-quality academic work. Part of achieving this quality is referencing in a way that is consistent and congruent with the requirements of the referencing style being used.

Therefore, inconsistent and/or incongruent referencing will result in a penalty of **a maximum of ten percent being deducted from the overall percentage** awarded to your assessment submission.

Please note that **evidence of plagiarism** in the form of copied or unreferenced work, absent reference lists, or exceptionally poor referencing **may result in action being taken in accordance with The IIE's Intellectual Integrity and Property Rights Policy (IIE023)**. Similarly, **evidence of excessive AI usage may result in action being taken in accordance with The IIE's Student Conduct, Discipline and Safety Policy (IIE015)**.

Markers are required to provide feedback to students by **circling/underlining the information in the table below that best describes the student's work and by adding constructive commentary where appropriate.** The examples provided are not exhaustive but illustrate the errors.

Deductions

- Where the student's work contains **five or more errors** aligned to the **minor errors column** below, **deduct 5% from the overall percentage.**
- Where the student's work contains **five or more errors** aligned to the **major errors column** below, **deduct 10% from the overall percentage.**
- **Where both minor and major errors** (e.g. two minor and three major, etc.) are present, **deduct 10% only** (and not 5% or 15%) from the overall percentage.

Required: **Consistent and congruent referencing**	**Minor errors** **Deduct 5% from overall percentage.** **Example: if the response receives 70%, deduct 5%. The final mark is 65%.**	**Major errors** **Deduct 10% from the overall percentage.** **Example: if the response receives 70%, deduct 10%. The final mark is 60%.**
Consistency • The **correct referencing style** for the **discipline** – i.e., either **Harvard**, OR **APA** (for Psychology)**,** OR **Law**, OR **IEEE** (for ICT/Engineering) – has been used consistently for all in-text references and in the bibliography/reference list. • **Concepts** and **ideas** that are **quoted** and/or **paraphrased** are referenced consistently throughout. • **Position of the in-text reference**: an in-text reference is positioned consistently where appropriate for every quote and paraphrase.	**Minor inconsistencies:** • The referencing style used is generally consistent with what is required, but there are one or two changes/errors in the format of in-text referencing and/or in the bibliography/reference list. • For example, page numbers for direct quotes in-text have been provided for one source, but not in another. Or, two book chapters in the bibliography/reference list have been referenced in two different formats. Or, the publication year has been placed after the author name in one bibliography/reference list entry, and after the source title in another, etc. • Concepts and ideas in quotes and/or paraphrases are typically referenced, but a full in-text reference is missing or incomplete from one or two small sections of the work. • Position of the references: in-text references are only given at the beginning and/or end of every paragraph.	**Major inconsistencies:** • Poor and wholly inconsistent referencing style used in-text and/or in the bibliography/reference list. • Multiple referencing styles for the same source types have been used. • For example, the format for direct quotes in-text and/or book chapters in the bibliography/reference list and/or year of publication in the bibliography/reference list is different across multiple instances. • Concepts and ideas in quotes and/or paraphrases are haphazardly referenced in-text. • Position of the references: in-text references are only given at the beginning or end of large sections of work.
Feedback on referencing consistency:		
Congruency • **Each source** reflected within **in-text references** is included accurately in the **bibliography/reference list.** • All **bibliography/reference list** entries are in the **required order** for the referencing style used (e.g. alphabetical, alphabetical under subheadings, numerical). • All **direct quotes** and **paraphrases** have been **integrated appropriately** into the text using introductory phrases, accurate grammar, etc.	**Minor incongruences:** • There is largely a match between the sources presented in-text and those in the bibliography/reference list, but one or two sources that appear in-text do not appear in the bibliography/reference list, or vice versa. Or key source information is missing from one or two in-text references or bibliography/reference list entries only (e.g. publication year, city of publication, URL date accessed, etc.). • There is a clear and largely accurate ordering of sources in the bibliography/reference list as required by the referencing style used, but with one or two references out of order. • An attempt has been made for source integration into the text using appropriate introductory phrases and grammar, but one or two quotes or paraphrases do not flow as clearly or logically within the sentence structure as they could.	**Major incongruences:** • No relationship/several incongruencies between the in-text referencing and the bibliography/reference list. • For example, multiple sources are included in-text, but not in the bibliography, and/or vice versa. Key source information is missing from multiple in-text references and/or reference list entries. A URL link, rather than the actual reference, is provided in the bibliography. Sources are repeated in the reference list, etc. • Most sources are listed in a haphazard order throughout the bibliography/reference list. • Few to no appropriate introductory phrases or rules of grammar have been applied, and many direct quotes and/or paraphrases feel disconnected from the flow of the text.
Feedback on referencing congruency:		
Overall feedback on referencing, with suggested improvements:		

Read this:

NB: In this section, we are going to ask you to make use of tools to complete a task that is not covered in your learning material. This has been included to help you explore and build features that you are not familiar with yet.

Anyone can code and create software; however, good software engineers create software that is testable, scalable, and maintainable. Good quality, clean code is a fine art that is not easy to achieve. We would like to start teaching you how to do this from day one.

This PoE will thus require that you not only create your solution but also to:

- Make use of version control (Git) – we will be using GitHub.
- Test your software using unit tests – we will be using JUnit.
- Automate these tests so that your code is tested with every change you make.

You will receive step-by-step instructions and video resources to help you make use of the above-mentioned tools.

What are we building?

Chat app

- **Message payload:** This contains the actual message text.
- **Message Sent:** A flag to indicate if the message has been sent.
- **Message Received:** A flag to indicate if the message has been received.
- **Message Read:** A flag that indicates if the message has been read.

It is also important to know who the message was sent to; this is often done by phone number. You can read more about chat applications here: https://quickblox.com/blog/beginners-guide-to-chat-app-architecture/

Finally:

This PoE is practical in nature and relies on the theoretical knowledge you will be gaining while completing the module, which covers the theoretical knowledge of the following coding constructs:

- Variables and variable scope
- Data types
- Classes, methods, and interfaces
- Operators
- Operator precedence
- Decisions
- Loops
- Arrays

You will practically implement each of these constructs in this PoE.

Instructions

You will need to make sure that you have the following to start this PoE.

1. A GitHub account (please use your Connect account to sign up) – you can sign up here: https://github.com/
2. You will be asked to write unit tests to ensure that your code is working correctly – please watch the following video to prepare: https://www.youtube.com/watch?v=MOhiM2SXZl0
3. The following is also needed to make use of Java Maven, so please use this link to help with automated testing in GitHub: https://www.youtube.com/watch?v=oz0Qd5H4Onk
4. Video on how to use GitHub Desktop: https://www.youtube.com/watch?v=bUgFv1Y5LJw

Each Submission:

1. Each part must be submitted correctly pushed to GitHub with the link submitted on Arc for each part: Part 1, Part 2 and PoE.
2. GitHub Requirements: Each part of the project should have a minimum of six commits. **You will lose 5% per task if you do not hand it in on GitHub.**
3. Presentation Requirement: You are required to produce a video presentation of your code for Part 1, Part 2 and the final PoE. This needs to include the full explanation and showing of your application running. Please note that you must also explain how your logic and flow are produced. This is worth marks, so do not leave it out. Please provide an unlisted YouTube video or a platform like that, which needs to include a voice-over of your own voice and no AI voices to be used.

All parts need to make use of a console application only; no GUIs are allowed, so no JOptionPane either.

Part 1 — Registration and login feature **(Marks: 100)**

At the end of this specific task, students should be able to:

- Create classes, methods, and other OOP programming constructs.
- Use decisions.
- Produce an application that accepts input and returns output.

Your very first task is to create a registration and login feature. This feature needs to allow users to read through the entire task before they start any work:

1. **Create an account by entering a username, password, and South African cell phone number.**
    a. The system needs to check that the following conditions are met and reply with the appropriate output message:

Conditions	**Messages**	
	True	**False**
Username contains an underscore and is no more than five characters long.	"Username successfully captured."	"Username is not correctly formatted; please ensure that your username contains an underscore and is no more than five characters in length."
Password meets the following password complexity rules; the password must be: • At least eight characters long. • Contain a capital letter. • Contain a number. • Contain a special character.	"Password successfully captured."	"Password is not correctly formatted; please ensure that the password contains at least eight characters, a capital letter, a number, and a special character."

b. Research to create a regular expression-based cell phone checker that will ensure that the cell phone number conforms to the following criteria:

Conditions	**Messages**	
	True	**False**
The cell phone number contains the international country code followed by the number, which is no more than ten characters long.	"Cell phone number successfully added."	"Cell phone number incorrectly formatted or does not contain international code."

Ensure that you attribute this code by providing a proper reference in your code.

2. **Login to the account using the same username and password.**

a. The system should provide the following messages to verify the user's authentication status:

Conditions	**Messages**	
	True	**False**
The entered username and password are correct, and the user is able to log in.	"Welcome <user first name>, <user last name> it is great to see you again."	"Username or password incorrect, please try again."

3. You will need to implement a **Login** class with the following methods to ensure that your application meets good coding standards and that the code you write is testable.

Method Name	Method Functionality
Boolean: checkUserName()	This method ensures that any username contains an underscore (_) and is no more than
Boolean: checkPasswordComplexity()	This method ensures that passwords meet the following password complexity rules: The password must be: • At least eight characters long. • Contain a capital letter. • Contain a number. • Contain a special character.
Boolean: checkCellPhoneNumber()	This method ensures that the cell phone is the correct length and contains the international country code.
String registerUser()	This method returns the necessary registration messaging indicating if: • The username is incorrectly formatted. • The password does not meet the complexity requirements. • The two above conditions have been met, and the user has been registered successfully.
Boolean loginUser()	This method verifies that the login details entered match the login details stored when the user registers.
String returnLoginStatus	This method returns the necessary messaging for: • A successful login • A failed login

4. It is good practice to never push code that has not been tested; you will need to create the following unit tests to verify that your methods are executing as expected:

Test: (assertEquals)	Test Data and expected system responses.
Username is correctly formatted:	**Test Data: "kyl_1"**
The username contains an underscore and is no more than five characters long.	**The system returns:** "Welcome <user first name> ,<user last name> it is great to see you."
Username incorrectly formatted:	**Test Data: "kyle!!!!!!!"**
	The system returns:

The username does not contain an underscore and is no more than five characters long.	"Username is not correctly formatted; please ensure that your username contains an underscore and is no more than five characters in length."
The password meets the complexity requirements.	**Test Data: "**Ch&&sec@ke99!"
	The system returns: "Password successfully captured."
The password **does not** meet the complexity requirements.	**Test Data: "**password"
	The system returns: **"**Password is not correctly formatted; please ensure that the password contains at least eight characters, a capital letter, a number, and a special character."
The cell phone is correctly formatted.	**Test Data:** +27838968976
	The system returns: "Cell number successfully captured."
The cell phone number is **incorrectly** formatted	**Test Data:** 08966553
	The system returns: "Cell number is incorrectly formatted or does not contain an international code; please correct the number and try again."
Test (assertTrue/False)	
Login Successful	**The system returns:** True
Login Failed	**The system returns:** False
Username correctly formatted	**The system returns:** True

Username incorrectly formatted.	**The system returns:** False
Password meets complexity requirements.	**The system returns:** True
Password does not meet complexity requirements.	**The system returns:** False
Cell phone number correctly formatted.	**The system returns:** True
Cell phone number incorrectly formatted.	**The system returns:** False

5. Watch the following video to help you create the necessary unit tests in NetBeans: https://www.youtube.com/watch?v=MOhiM2SXZl0

Make sure to use the test data detailed in the table for assertEquals, as this will be used to mark your task.

Part 2 — Sending Messages (Marks: 100)

At the end of this specific task, students should be able to:

- *Create and work with loops*
- *Handle and manipulate strings*
- *(Learning Units 4 and 5).*

It is good practice to leave your main branch as your live long branch; this means that the code on this branch is always in perfect working order and tested. We make use of feature branches in order to ensure that any code we push to GitHub does not break our main branch. You can create a feature branch by running the following command:

Git checkout -b KhanbanTasks (you can use any branch name)

You are welcome to make use of GitHub desktop or your IDE to push code to GitHub if you are not comfortable with using the command line.

You can now add the following functionality to your application:

1. The users should only be able to send messages if they have logged in successfully.
2. The applications must display the following welcome message: "**Welcome to QuickChat."**
3. The user should then be able to choose one of the following features from a numeric menu:
    a. Option 1) Send Messages
    b. Option 2) Show recently sent messages – this feature is still in development and should display the following message: **"Coming Soon."**
    c. Option 3) Quit
4. The application should run until the user selects 'quit' to exit.
5. Users should define how many messages they wish to enter when the application starts; the application should allow the user to enter only the set number of messages.
6. Each message should contain the following information:

Unique Message ID for tracking	This is a randomly generated ten-digit number that is stored for each correlating message in order to retrieve or delete messages.
Num messages sent	This value starts with the number; this number is incremented and autogenerated as more messages are sent.
Recipient	This value contains the cell number of the recipient and ensures that the number has no more than ten characters and contains an international code.
Message	A short message, which **should not exceed 250 characters** in length. The following error message should be displayed if the task description is too long: ***"Please enter a message of less than 250 characters."*** *OR* ***"Message sent"*** if the message description meets the requirements.
Message Hash	The system must autogenerate a Message Hash, which contains the first two numbers of the message ID, a colon(:), the number of the message(:), and the first and last words in the message. The hash should be displayed in all caps: 00:0:HITHANKS

Send Message	Once the message is completed, the system should ask the user to choose one of the following options:
	• Send Message
	  ○ "Message successfully sent"
	• Disregard Message
	  ○ "Press 0 to delete the message"
	• Store Message to send later
	  ○ "Message successfully stored"

Store Message in a JSON file	Research

7. The full details of each message should be displayed on the screen after it has been sent and should show all the information requested in the table above in the following order: Message ID, Message Hash, Recipient, Message.
6. The total number of messages should be accumulated and displayed once all the messages have been sent.

Create a **Message class** that contains the following messages:

Method Name	**Method Functionality**
Boolean: checkMessageID()	This method ensures that the message ID is not more than ten characters.
String: checkRecipientCell()	This method ensures that the recipient cell number is no more than ten characters long and starts with a code.
String: createMessageHash()	This method creates and returns the Message Hash.
String:SentMessage()	This method should allow the user to choose if they want to send, store, or disregard the message.
String: printMessages()	This method returns all the messages sent while the program is running.
Int: returnTotalMessagess()	This method returns the total number of messages sent.
Your own defined storeMessage() method	Research how to create a method to store the messages in JSON.

8. Please use the following test data to create unit tests.

Test Data:	
Num Messages	2
Test Data for Task 1	
Num sent messages	Auto generated.
Message Hash	Auto generated.
MessageID	Auto generated
Recipient Number	+27718693002
Message	“Hi Mike, can you join us for dinner tonight?”
SendMessage	Select Send
Return total number sent	Auto Generated

Test Data for Message 2	
Num sent messages	Auto generated.
Message Hash	Auto generated.
MessageID	Auto generated
Recipient Number	08575975889
Message	"Hi Keegan, did you receive the payment?"
SendMessage	Select Discard
Return total number sent	Auto generated

9. Create the following unit tests:

Test AssertEquals:	
The message should **not be** more than 250 Characters	Test for both success and failure. **<u>The system should return:</u>** **<u>Success</u>** ***"Message ready to send."*** **<u>Failure:</u>** ***"Message exceeds 250 characters by X [enter number here]; please reduce the size."***

The recipient number is correctly formatted	**<u>The system should return:</u>** **<u>Success:</u>** ***"Cell phone number successfully captured."*** **<u>Failure:</u>** ***"Cell phone number is incorrectly formatted or does not contain an international code. Please correct the number and try again."*** **HINT: TRY AND REUSE THE METHOD AND TESTS YOU USED IN THE LAST TASK.**
Message hash is correct.	**<u>The system should return:</u>** <u>00:0:HITONIGHT</u> when supplied with the data from Test Case 1. The system should test the remainder of the message hashes in a loop (refer to video):

Message ID is created	**The system should return:** Message ID generated: <Message ID>
MessageSent	**Additional test data:** 1): User selected 'Send Message' 2): User selected 'Disregard Message' 3): User selected 'Store Message'
	The system should return: 1) "Message successfully sent." 2) "Press 0 to delete the message." 3) "Message successfully stored."

7. Finally, developers make use of Continuous Integration and Continuous Deployment (CI/CD) pipelines to iteratively build systems and to test not only the functionality but also the quality of their code. It is good practice to start working with at least CI in mind. We will be implementing GitHub actions to:
    a. Automate the tests we have written to run whenever we update our code.
    i. Make sure you have signed up for the GitHub student developer pack.
    ii. Follow the steps detailed below to automate your tests using GitHub Actions: https://youtu.be/oz0Qd5H4Onk

Part 3 — Store Data and Display Task Report (Marks: 100)

At the end of this specific task, students should be able to:

- Handle and manipulate strings.
- Create and work with arrays.

You will now add the final features to your app, write and automate the unit tests, and submit your final project. You need to get everything from Part 1 and Part 2 working and extend your application to allow for the following:

1. Users should be able to populate the following arrays (no hard-coding):

Array	**Contents**
Sent Message	Contains all the messages sent.
Disregarded Messages	Contains all the messages that were disregarded.
Stored Messages	Use online sources to help you read the JSON file you stored into an array that contains the stored messages.
Message Hash	Contains all the message hashes.
Message ID	Contains all the message IDs.

2. Create a fourth main menu option for “Stored Messages”; users should be able to use these arrays to:
    a. Display the sender and recipient of all stored messages.
    b. Display the longest stored message.
    c. Search for a message ID and display the corresponding recipient and message.
    d. Search for all the messages stored for a particular recipient.
    e. Delete a message using the message hash.
    f. Display a report that lists the full details of all the stored messages.

3. Use the following test data for your unit tests and to populate your arrays:

Test Data Message 1	
Recipient	+27834557896
Message	“Did you get the cake?”
Flag	Sent

Test Data Message 2	
Recipient	+27838884567
“Message”	“Where are you? You are late! I have asked you to be on time.”
Flag	Stored
Test Data Message 3	
Recipient	+27834484567
“Message”	“Yohoooo, I am at your gate.”
Flag	Disregard

Test Data Message 4	
Developer	0838884567
“Message”	“It is dinner time !”
Flag	Sent

Test Data Message 5	
Recipient	+27838884567
“Message”	“Ok, I am leaving without you.”
Flag	Stored

4. Create the following unit tests:

Test: (assertEquals)	**Test Data and expected system responses.**
Sent Messages array correctly populated: The Messages array contains the expected test data.	**Test Data: Developer entry for Test data for message 1-4**
	The system returns: "Did you get the cake?", "It is dinner time!"
Display the longest Message	**Test Data: message 1-4**
	The system returns: "Where are you? You are late! I have asked you to be on time."
Search for messageID	**Test Data: message 4**
	"0838884567",
	The system returns: ""It is dinner time!"
Search all the messages sent or stored regarding a particular recipient	**Test Data:** +27838884567
	The system returns: "Where are you? You are late! I have asked you to be on time." "Ok, I am leaving without you."
Delete a message using a message hash	**Test Data:** Test Message 2
	The system returns: Message: "Where are you? You are late! I have asked you to be on time" successfully deleted.
Display Report	**The system returns a report that shows all the sent messages, including the:** **Message Hash** **Recipient** **Message**

Assessment Sheet (Marking Rubric)

Please note: Tear off this section and **attach** it to your work when you submit it/ If this is an online submission, then this information needs to be included in the online submission.

MODULE NAME:	**MODULE CODE:**
PROGRAMMING 1A	**PROG5121/p/w**

STUDENT NAME:	
STUDENT NUMBER:	

Part 1	**Levels of Achievement**			**Student Result**	**Feedback**
In order to be awarded full marks for these elements of Part 1, students need to have:	**Excellent**	**Good**	**Developing**		
	Score Ranges Per Level (½ marks possible)				
Registration Feature: * Username contains an underscore and is no more than five characters long.	**13 – 15** Feature excellently implemented.	**8 – 12** Feature working mostly with some bugs**.**	**0 – 7** Feature not implemented or very buggy, or the app does not compile.		
Registration Feature: * Password meets complexity requirements.	**13 – 15** Feature excellently implemented.	**8 – 12** Feature working mostly with some bugs**.**	**0 – 7** Feature not implemented or very buggy, or the app does not compile.		
Registration Feature: * Research regex cell phone checker is implemented.	**13 – 15** Feature excellently implemented and attributed.	**8 – 12** Feature working mostly, with some bugs or is not attributed.	**0 – 7** Feature not implemented or very buggy, or the app does not compile. Feature is not attributed.		

Login Feature: ***** Appropriate decision structure is used to verify the user Authentication.	**13 – 15** Feature excellently implemented.	**8 – 12** Feature working mostly with some bugs**.**	**0 – 7** Feature not implemented or very buggy, or the app does not compile.		
Login Feature: ***** The system responds with appropriate confirmation and error messages.	**13 – 15** Feature excellently implemented.	**8 – 12** Feature working mostly with some bugs**.**	**0 – 7** Feature not implemented or very buggy, or the app does not compile.		
Unit Tests: ***** Unit tests are created and correctly tests for functionality.	**13—15** Feature excellently implemented.	**8—12** Feature working mostly with some bugs**.**	**0—7** Feature not implemented or very buggy, or the app does not compile.		
Coding Standard and Code Complexity	**7—10** Good variable names, low complexity, no redundant code, commented class files, efficient code.	**4-7** Sufficient variable names, acceptable complexity, low code redundancy, comments present, leans to towards efficient code.	**0—3** Poor variable names, code is overly complex, redundant code present, poor commenting, code is not efficient.		
GitHub not used **No use of GitHub or incorrect use of GitHub**	**-5%**				

MODULE NAME:	MODULE CODE:
PROGRAMMING 1A	PROG5121/p/w

STUDENT NAME:	
STUDENT NUMBER:	

Part 2	Levels of Achievement			Student Results	Feedback
In order to be awarded full marks for these elements of Part 2, students need to have:	Excellent Score Ranges Per Level (½ marks possible)	Good	Developing		
Welcome Message and Menu Feature: Welcome message displays correctly. Menu option displays correctly.	**4—5** A numeric menu is displayed which allows the user to choose an option using the numbers 1-3.	**2—3** A numeric menu is displayed; users are able to choose an option.	**0—1** The menu does not display (0) or throws errors when the user tries to enter a numeric option (1 max).		
Add Message Feature: While loop	**4—5** The application loop runs correctly and quits on correct input.	**2—3** The loop is present but does not quit on correct input (2 max) or does not run for the duration of the application.	**0—1** No loop present (0), infinite loop (1).		

Add Message Feature: For Loop	**8 – 10** A for loop is used to allow the user to enter the assigned number of messages. The loop runs and exits correctly.	**4 – 7** A for loop is used for one-off errors, still allows the user to enter messages.	**0 – 3** No loop (0) Infinite loop (1)		
Send Message: Enter message details	**8 – 10** The application provides adequate output which allows the user to enter the necessary message details. Appropriate variables exist.	**4 – 7** The feature allows the user to add details to messages.	**0 – 3** Not all data can be entered, the variables are not stored correctly, etc.		
Add Message: Message Number created in loop (using loop counter)	**4 – 5** The message number increments correctly as the loop runs.	**2 – 3** One-off errors.	**0 – 1** The loop does not run correctly and does not generate task numbers.		
Add Message: Message ID displays correctly and created with string manipulation (substring)	**8 – 10** The Message ID is created using string manipulation and loop counters and is displayed correctly for all the looped tests.	**4 – 7** The Message ID is created using string manipulation and counters but contains one-off or other errors.	**0 – 3** MessageID is hard coded or incorrect.		
Add Message: Message Hash displays correctly and is created with string manipulation	**8 – 10** The Message Hash is created using string manipulation and loop counters and is displayed correctly for all the looped tests.	**4 – 7** The Message Hash is created using string manipulation and counters but contains one-off or other errors.	**0 – 3** Message Hash is hard coded or incorrect.		

Add Message: Message details display correctly	**8 – 10** The messages details are displayed in the correct order and the tests pass.	**4 – 7** Majority of the message details displays correctly.	**0 – 3** Message details do not display (0) or display with multiple errors (1).		
Message is stored as JSON File	**8 – 10** Messages are successfully stored as a JSON file and the code is attributed.	**4 – 7** Messages are stored as a JSON file and the code is not attributed.	**0 – 3** Messages are not stored as JSON file or the code contains multiple errors (1).		
Unit Tests	**8—10** Unit tests are created, passed and adequately test the functionality.	**4—7** Unit tests are created but do not adequately test the functionality**.**	**0—3** No Test (0), very little tests with errors in the test class (1).		
Automated Unit Tests	**4—5** TestJava. yml file is updated to run Task Class tests and executes in GitHub. All tests are present and passed.	**2—3** ThestJava.yml is created but does not contain all tests for the Task class. Executes in GitHub with some test failures.	**0—1** No tests (0). Multiple Errors (1).		
Coding Standard and Code Complexity	**7—10** Good variable names, low complexity, no redundant code, commented class files, efficient code.	**4-7** Sufficient variable names, acceptable complexity, low code redundancy, comments present, leans towards efficient code.	**0—3** Poor variable names, code is overly complex, redundant code present, poor commenting, code is not efficient.		
GitHub not used **No use of GitHub or incorrect use of GitHub**	**-5%**				

MODULE NAME:	MODULE CODE:
PROGRAMMING 1A	PROG5121/p/w

STUDENT NAME:	
STUDENT NUMBER:	

PoE	Levels of Achievement			Student Results	Feedback
In order to be awarded full marks for these elements of the final PoE, students need to have:	Excellent	Good	Developing		
	Score Ranges Per Level (½ marks possible)				
Arrays correctly populated	**8 – 10** Arrays are created and populated correctly.	**4 – 7** Arrays are created but not populated correctly.	**0 – 3** No arrays (0). Arrays with multiple errors (1).		
Display details for the longest message	**4—5** The application successfully searches the parallel arrays and displays the correct output.	**2—3** The system searches parallel arrays; incorrect output is displayed.	**0—1** Feature omitted (0). Feature contains multiple errors (1).		
Search Array for messages sent to particular recipient	**8 - 10** The application successfully searches the parallel arrays and displays the correct output.	**4 – 7** The system searches parallel arrays; incorrect output is displayed.	**0 – 3** Feature omitted (0). Feature contains multiple errors (1).		

Delete Message using message Hash	**8 - 10** Array is successfully searched, and the appropriate value is removed.	**4 – 7** Array is searched, incorrect values or no value are removed.	**0 – 3** Feature not implemented (0) or contains multiple errors (1).		
Read JSON file into an array and display it to the user	**8 – 10** File is read in correctly and the code is attributed.	**4 – 7** File is read in and code is not attributed.	**0 – 3** Feature not implemented (0) or with multiple errors (1).		
Display Message Report	**8 – 10** Report displays correctly with the necessary information.	**4 – 7** Report displays but omits some information.	**0 – 3** Feature not implemented (0) or with multiple errors (1).		
Unit Tests	**12-15** Unit tests are created, passed and adequately test the functionality.	**7-11** Unit tests are created but do not adequately test the functionality.	**0—5** No Test (0), very little tests with errors in the test class (1).		.
Automated Unit Tests	**4-5** TestJava. yml file is updated to run Task Class tests and executes in GitHub. All tests are present and passed.	**2-3** ThestJava.yml is created but does not contain all tests for the Task class. Executes in GitHub with some test failures.	**0—1** No tests (0). Multiple Errors (3).		
Completed application functions as whole	**12-15** Exceptional work is submitted that shows that the student went beyond what is expected.	**7-11** Every feature detailed in tasks 1-3 is working together holistically.	**0-5** No features are working or multiple errors are present.		

Coding Standard and Code Complexity	**7—10** Good variable names, low complexity, no redundant code, commented class files, efficient code.	**4-7** Sufficient variable names, acceptable complexity, low code redundancy, comments present, leans to towards efficient code.	**0—3** Poor variable names, code is overly complex, redundant code present, poor commenting, code is not efficient.		
GitHub not used **No use of GitHub or incorrect use of GitHub**	**-5%**				

[TOTAL MARKS: 300]